

Enumerazioni (enum)

Enumerazioni (enum)

Come usare le enumerazioni in C++ per definire insiemi di costanti con nome.

Cos'è un enum?

Supponi di dover rappresentare i giorni della settimana nel tuo programma. Potresti usare i numeri 0, 1, 2... ma chi si ricorda che 3 è giovedì?

Un'enumerazione (`enum`) ti permette di dare nomi significativi a un insieme fisso di valori. Invece di `3`, scrivi `GIOVEDI`. Il codice diventa leggibile come l'italiano.

Definire un `enum`

```
enum NomeEnum {  
    VALORE1,  
    VALORE2,  
    VALORE3  
};
```

Esempio concreto:

```
enum GiornoSettimana {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
};
```

Per convenzione, i valori si scrivono tutti in **MAIUSCOLO**.

Usare un **enum**

```
GiornoSettimana oggi = MERCOLEDI;    // dichiara una variabile di tipo
GiornoSettimana

if (oggi == SABATO || oggi == DOMENICA) {
    cout << "Weekend!" << endl;
} else {
    cout << "Giorno lavorativo." << endl;
}
```

I valori numerici nascosti

Internamente, ogni valore dell'enum è un numero. Per default, il primo vale 0, il secondo 1, e così via:

```
enum Stagione {
    PRIMAVERA,    // 0
    ESTATE,       // 1
    AUTUNNO,      // 2
    INVERNO       // 3
};

Stagione s = ESTATE;
cout << s << endl;    // stampa 1 (il numero interno)
```

Puoi assegnare valori specifici:

```
enum Mese {
    GENNAIO = 1,    // parte da 1 invece di 0
    FEBBRAIO,       // 2
    MARZO,           // 3
    // ...
    DICEMBRE = 12
};

Mese m = MARZO;
cout << m << endl; // 3
```

Abbinare enum e switch

`enum` e `switch` si abbinano perfettamente — il codice è chiaro e privo di ambiguità:

```
enum Semaforo {
    ROSSO,
    GIALLO,
    VERDE
};

void istruzione(Semaforo s) {
    switch (s) {
        case ROSSO:
            cout << "STOP!" << endl;
            break;
        case GIALLO:
            cout << "Attenzione, rallentare." << endl;
            break;
        case VERDE:
            cout << "Via libera!" << endl;
            break;
    }
}

int main() {
    istruzione(ROSSO);    // STOP!
    istruzione(VERDE);    // Via libera!
    return 0;
}
```

enum class : la versione moderna e più sicura

Il C++ moderno introduce `enum class`, che evita un problema degli enum classici: i nomi possono andare in conflitto tra enum diversi.

```
// Problema con enum classico:
enum Colore { ROSSO, VERDE, BLU };
enum Frutto { ROSSO, VERDE, GIALLO }; // ERRORE: ROSSO è già definito!

// Soluzione con enum class:
enum class Colore { ROSSO, VERDE, BLU };
enum class Frutto { ROSSO, VERDE, GIALLO }; // nessun conflitto!

int main() {
    Colore c = Colore::ROSSO; // devi specificare il nome dell'enum
    Frutto f = Frutto::ROSSO; // completamente separato da Colore::ROSSO

    if (c == Colore::ROSSO) {
        cout << "Il colore è rosso" << endl;
    }
    return 0;
}
```

Con `enum class` devi sempre scrivere `NomeEnum::VALORE` — è più lungo, ma non ci sono mai ambiguità. È la scelta preferita nel C++ moderno.

Convertire tra enum e numeri

```
enum Livello { FACILE, MEDIO, DIFFICILE };

Livello l = MEDIO;
int n = static_cast<int>(l); // da enum a numero → 1
cout << n << endl;

Livello l2 = static_cast(2); // da numero a enum → DIFFICILE
```

Esempio pratico: sistema di voti

```
#include <iostream>
using namespace std;

enum class Giudizio {
    OTTIMO,
    BUONO,
    SUFFICIENTE,
    INSUFFICIENTE
};

// Converte un voto numerico nel giudizio corrispondente
Giudizio classificaVoto(int voto) {
    if (voto >= 9) return Giudizio::OTTIMO;
    if (voto >= 7) return Giudizio::BUONO;
    if (voto >= 6) return Giudizio::SUFFICIENTE;
    return Giudizio::INSUFFICIENTE;
}

// Stampa il giudizio in parole
void stampaGiudizio(Giudizio g) {
    switch (g) {
        case Giudizio::OTTIMO:      cout << "Ottimo!" << endl;
        break;
        case Giudizio::BUONO:       cout << "Buono" << endl;
        break;
        case Giudizio::SUFFICIENTE: cout << "Sufficiente" << endl;
        break;
        case Giudizio::INSUFFICIENTE: cout << "Insufficiente" << endl;
        break;
    }
}

int main() {
    int voti[] = {9, 7, 5, 10, 6};
    for (int voto : voti) {
        cout << "Voto " << voto << ": ";
        stampaGiudizio(classificaVoto(voto));
    }
    return 0;
}
```

Output:

Voto 9: Ottimo!
Voto 7: Buono
Voto 5: Insufficiente
Voto 10: Ottimo!
Voto 6: Sufficiente